

1 Architektura logiczna

1.1 Pakiet `com.els.annotation`

Secure adnotacja umieszczana na metodach, włączająca mechanizm Entity Level Security dla wskazanej encji i akcji.

1.2 Pakiet `com.els.aspect`

SecurityAspect aspekt Spring AOP przechwytyjący metody oznaczone adnotacją `@Secure`, pobierający informacje o użytkowniku z kontekstu bezpieczeństwa oraz aktywujący odpowiednie filtry Hibernate.

1.3 Pakiet `com.els.context`

SecurityContext przechowuje informacje o aktualnie uwierzytelnionym użytkowniku w kontekście wykonywanego wątku.

1.4 Pakiet `com.els.cache`

PermissionCache mechanizm buforowania wyników rozwiązywania uprawnień dla kombinacji użytkownik-encja-akcja.

PermissionKey klucz cache identyfikujący kontekst zapytania bezpieczeństwa.

1.5 Pakiet `com.els.domain`

User reprezentuje użytkownika systemu wraz z przypisanymi rolami.

Role abstrakcyjna definicja roli bezpieczeństwa.

SimpleRole prosta rola bez zależności hierarchicznych.

CompositeRole rola złożona, umożliwiającą budowanie hierarchii ról.

Permission encja definiująca politykę dostępu do encji dla użytkownika lub roli.

Action enumeracja określająca typ operacji (SELECT, UPDATE, DELETE, INSERT).

AccessType enumeracja określająca typ dostępu (WHITELIST, BLACKLIST).

1.6 Pakiet `com.els.repository`

PermissionRepository repozytorium dostępu do polityk bezpieczeństwa zapisanych w bazie danych.

UserRepository repozytorium użytkowników systemu.

RoleRepository repozytorium ról bezpieczeństwa.

1.7 Pakiet `com.els.service`

PermissionResolver centralny komponent odpowiedzialny za rozwiązywanie efektywnych uprawnień użytkownika na podstawie przypisanych ról i polityk dostępu.

1.8 Pakiet `com.els.strategies`

AccessStrategy interfejs strategii budowania warunków ograniczających dostęp do danych.

WhitelistStrategy strategia tworząca warunek dostępu oparty na liście dozwolonych identyfikatorów.

BlacklistStrategy strategia tworząca warunek dostępu oparty na liście zabronionych identyfikatorów.

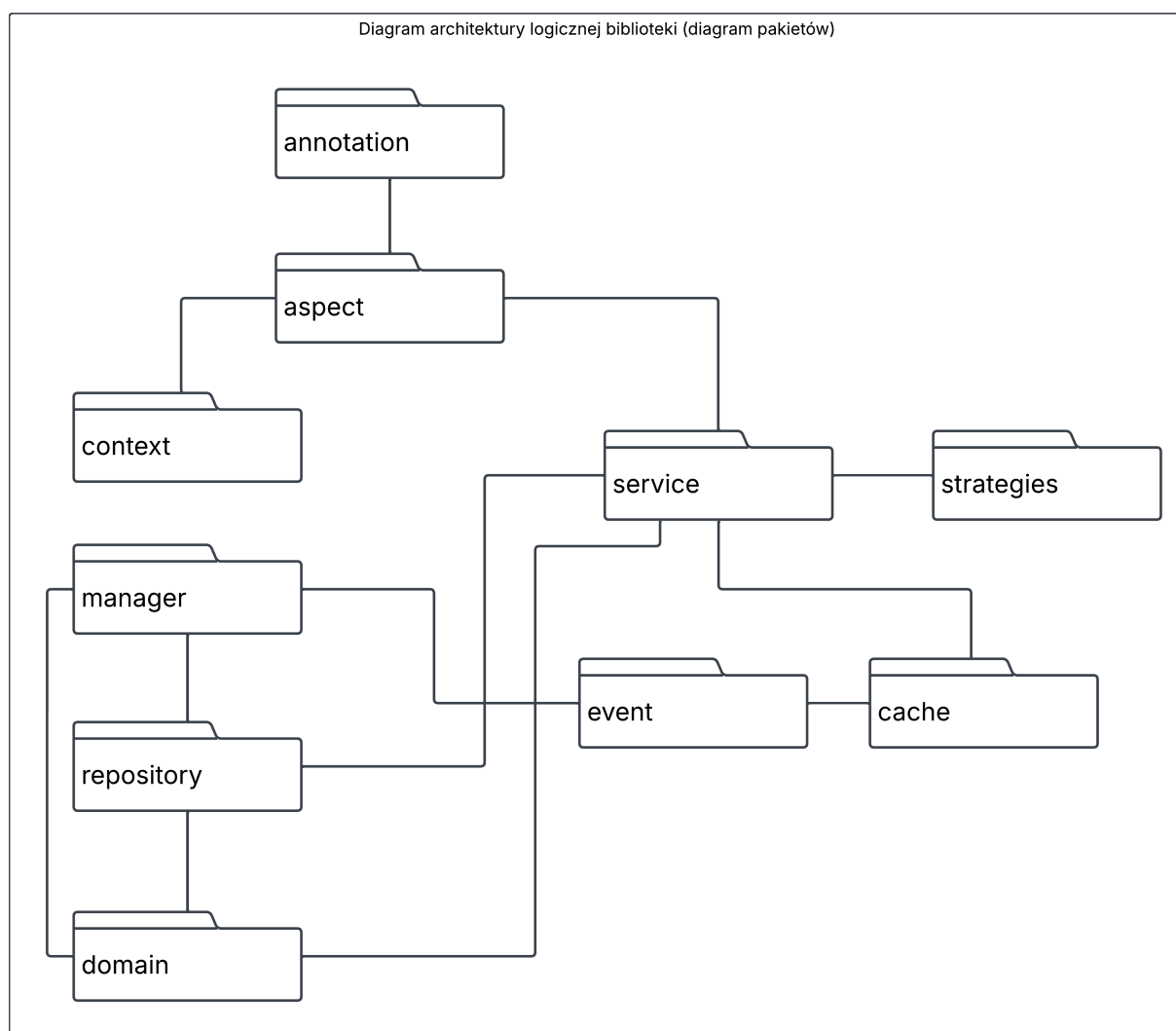
1.9 Pakiet `com.els.manager`

PermissionManager komponent zarządzający cyklem życia polityk dostępu oraz publikujący zdarzenia o ich zmianach.

1.10 Pakiet `com.els.event`

PermissionUpdateEvent zdarzenie informujące o zmianie polityki dostępu.

CacheInvalidator nasłuchuje zdarzeń zmian uprawnień i odpowiada za unieważnianie cache.



Rysunek 1: Diagram zależności pakietów biblioteki Entity Level Security (ELS)

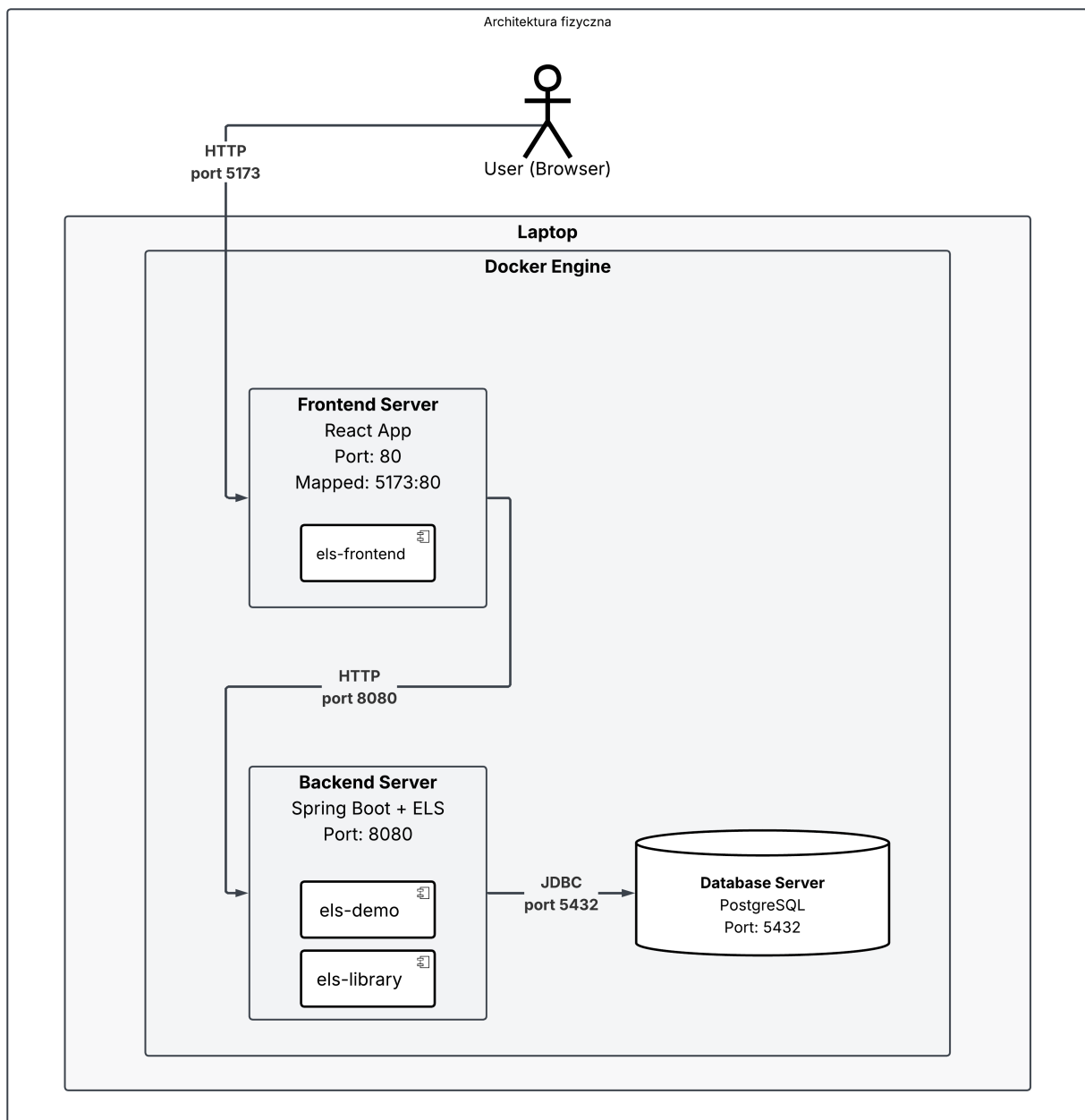
2 Architektura fizyczna

Architektura fizyczna systemu opiera się na trójwarstwowym modelu wdrożeniowym z wykorzystaniem konteneryzacji Docker. Wszystkie komponenty systemu uruchamiane są na jednej maszynie fizycznej (komputerze deweloperskim), natomiast Docker pełni rolę warstwy uruchomieniowej zapewniającej izolację poszczególnych usług. Poszczególne komponenty systemu zostały uruchomione w oddzielnych kontenerach, co ułatwia wdrażanie i utrzymanie rozwiązania.

Warstwa kliencka obejmuje przeglądarkę internetową użytkownika, która komunikuje się z serwerem frontendowym za pośrednictwem protokołu HTTP. Frontend, oparty o aplikację React, działa w kontenerze Docker i udostępnia interfejs użytkownika, przekazując żądania do warstwy backendowej.

Warstwa backendowa została zaimplementowana jako aplikacja Spring Boot, uruchamiana w osobnym kontenerze Docker i nasłuchująca na porcie 8080. Backend zawiera zarówno logikę aplikacyjną systemu, jak i bibliotekę Entity Level Security (ELS), odpowiedzialną za egzekwowanie reguł dostępu do danych.

Warstwa danych realizowana jest przez serwer bazy danych PostgreSQL, również uruchomiony w kontenerze Docker.



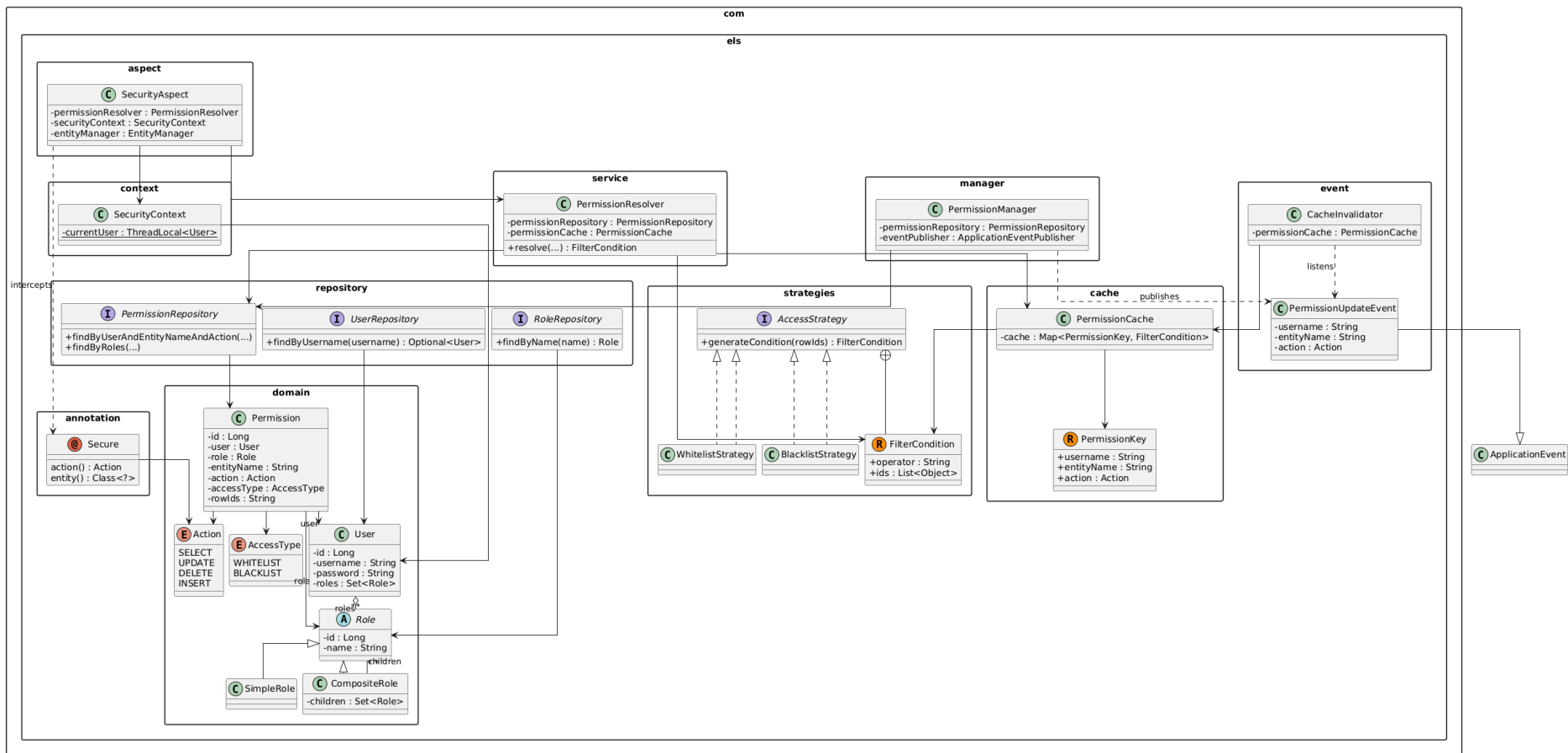
Rysunek 2: Diagram architektury fizycznej systemu ELS

3 Diagramy klas

W niniejszym rozdziale przedstawiono diagramy klas dla dwóch odrębnych artefaktów składających się na realizowane rozwiązanie: biblioteki Entity Level Security (ELS) oraz aplikacji demonstracyjnej MediSec. Diagramy te ilustrują strukturę statyczną systemu, ukazując kluczowe klasy oraz relacje pomiędzy nimi.

3.1 Biblioteka ELS

Diagram klas biblioteki ELS prezentuje główne elementy modelu domenowego bezpieczeństwa oraz komponenty odpowiedzialne za rozwiązywanie i egzekwowanie polityk dostępu do danych na poziomie encji. Diagram koncentruje się na klasach tworzących rdzeń biblioteki i nie obejmuje elementów aplikacji demonstracyjnej ani warstwy prezentacji.

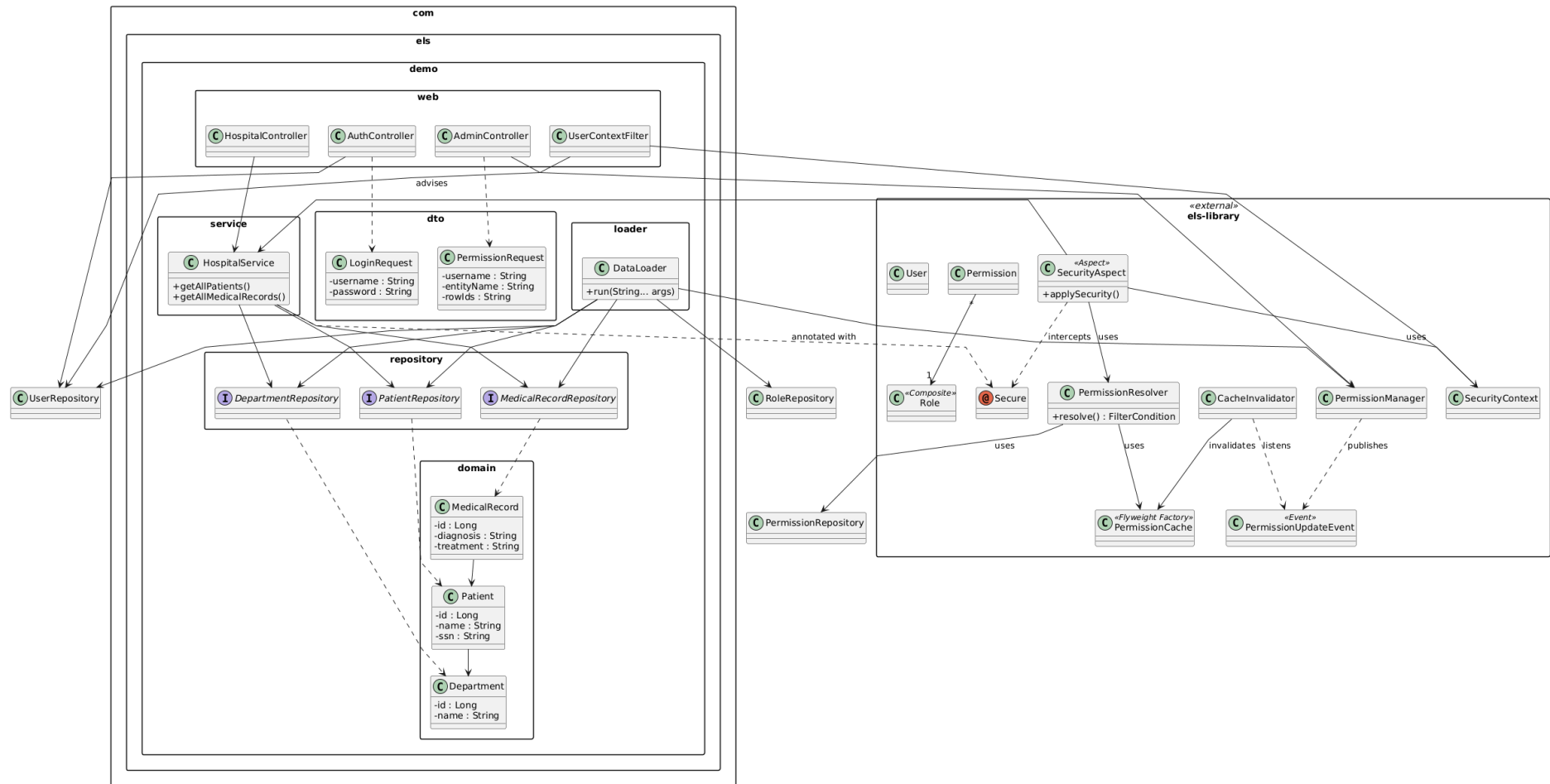


Rysunek 3: Diagram klas biblioteki ELS

3.2 Aplikacja demo

Aplikacja demonstracyjna MediSec została opracowana w celu zaprezentowania praktycznego zastosowania biblioteki ELS w realistycznym kontekście aplikacji biznesowej. Symuluje ona uproszczony system zarządzania danymi szpitalnymi, obejmujący przykładowe encje domenowe, takie jak pacjenci, lekarze oraz wizyty.

Celem aplikacji demo nie jest dostarczenie pełnej funkcjonalności systemu szpitalnego, lecz stworzenie środowiska umożliwiającego obserwację wpływu polityk bezpieczeństwa na dostęp do danych. Diagram klas aplikacji MediSec ilustruje strukturę tej aplikacji oraz zależności pomiędzy jej podstawowymi elementami, stanowiąc punkt odniesienia dla analizy integracji z biblioteką ELS.



Rysunek 4: Diagram klas aplikacji demonstracyjnej – widok szczegółowy

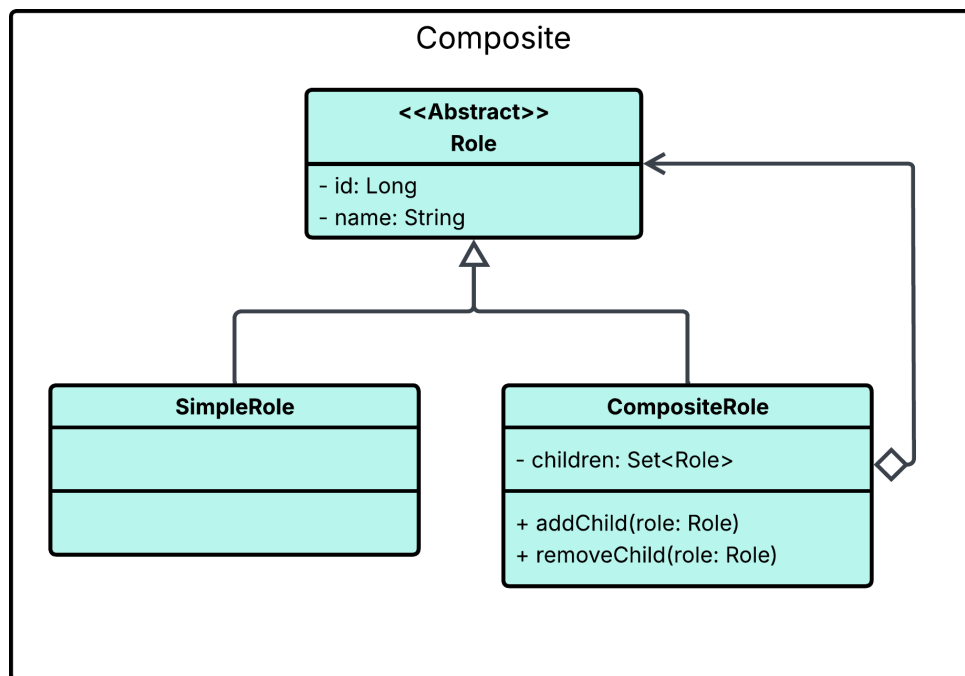
4 Wzorce projektowe

4.1 Kompozyt (Composite Pattern)

4.1.1 Cel wzorca

Wzór stosujemy aby traktować pojedyncze proste role oraz grupy(role złożone) w ten sam sposób. Pozwala to na tworzenie struktur drzewiastych. Przypisując uprawnienia do roli-dziecka (np. LEKARZ) są one przekazywane do wszystkich ról-rodziców (np. KARDIOLOG, ORDYNATOR). Biblioteka potem agreguje wszystkie uprawnienia do celu wyznaczenia ostatecznych końcowych uprawnień. Ponadto sprawia to, że można dynamicznie dodawać nowe role i zapobiega powtarzaniu przypisywania wszystkich uprawnień dla każdej roli z osobna.

4.1.2 Diagram wzorca



Rysunek 5: Diagram Kompozytu

4.1.3 Konsekwencje wzorca

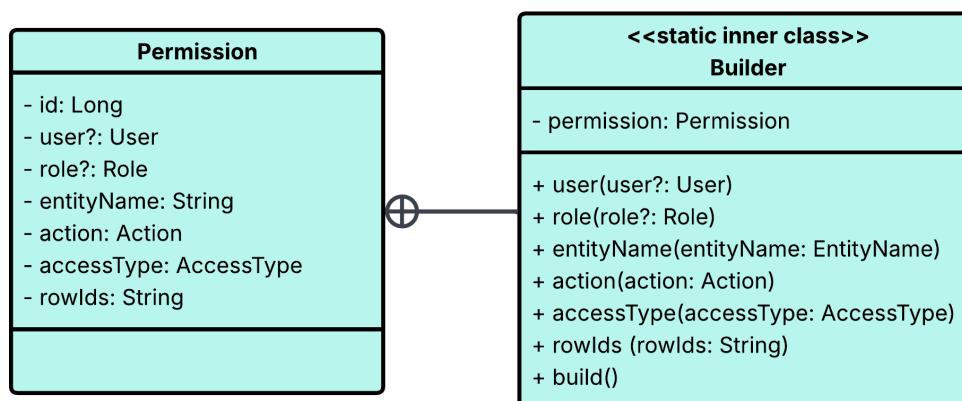
- Zastosowanie wzorca wymaga rekurencyjnego przechodzenia przez drzewo ról, co w przypadku głębokiej struktury może być **kosztowne obliczeniowo i pamięciowo**.
- **Ryzyko cyklicznych zależności** – jeśli administrator przez pomyłkę zdefiniuje role tak, że: Rola A zawiera Rolę B, a Rola B zawiera Rolę A to powstanie nieskończona pętla. Przy próbie odczytu rekurencji powstanie `StackOverflowError`.
- Utrudnienie debugowania – zamiast prostych pojedynczych ról mamy sieć powiązań. W trakcie debugowania ciężiej jest stwierdzić dlaczego dany użytkownik ma konkretne uprawnienie.

4.2 Builder

4.2.1 Cel wzorca

Podczas nadawania uprawnień tworzone jest wiele obiektów klasy `Permission`. Jej konstruktor jest złożony i może zawierać pola `null`. Wzorec Builder sprawia, że tworzenie obiektu staje się czytelne i samo dokumentujące się. Dzięki temu nie musimy tworzyć wielu konstruktorów (np. oddzielnego gdy `user` jest `null` i oddzielnego gdy `role` jest `null`).

4.2.2 Diagram wzorca



Rysunek 6: Diagram Buildera

4.2.3 Konsekwencje wzorca

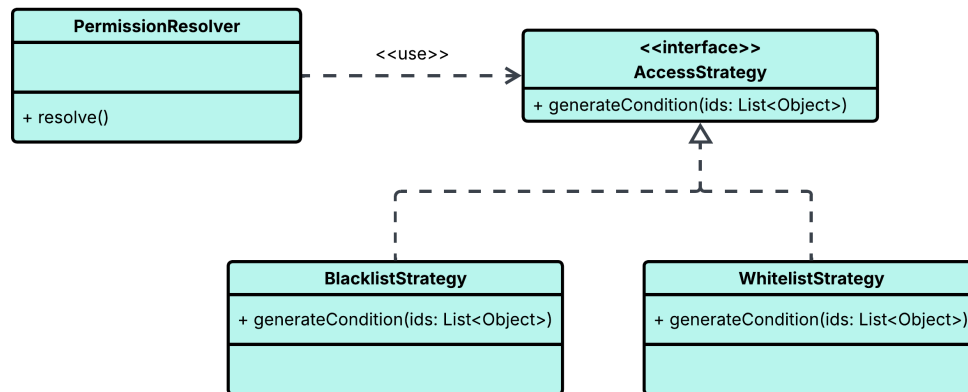
- **Nadmiarowy kod** (wesja bez zewnętrznych bibliotek np. Lombok) – powoduje znaczny rozrost klasy `Permission`. Kod staje się dłuższy o kilkadziesiąt linii przez co klasa może wydawać się bardziej złożona niż w rzeczywistości jest.

4.3 Strategia (Strategy)

4.3.1 Cel wzorca

W bibliotece ELS problemem jest różny sposób interpretacji listy `ID` w zależności od tego `WHITELIST` czy `BLACKLIST`. Użycie Strategy eliminuje nadmierne instrukcje warunkowe i sprawia, że kod staje się bardziej czytelny. Ponadto otwiera to możliwość rozszerzenia biblioteki o nowy typ zabezpieczenia (Open/Closed Principle).

4.3.2 Diagram wzorca



Rysunek 7: Diagram Strategy

4.3.3 Konsekwencje wzorca

- **Zwiększona liczba klas** – pomimo że pozbywamy się nadmiernych instrukcji warunkowych to musimy dodać nowe klasy do struktury.
- **Sztywność parametrów interfejsu** – istnieje jedna metoda `generateCondition()`, która przyjmuje listę id. Jeśli w przyszłości ktoś chciałby dodać strategię przyjmującą inne argumenty to zmusiłoby go to do zmiany interfejsu.
- **Zbytняя komplikacja kodu** – strategia jest "eleganckim" rozwiązaniem, ale znacznie komplikuje ona kod biblioteki. Ten sam efekt można by było osiągnąć przez zastosowanie kilku warunków `if else` (choć nie byłoby to takie schludne rozwiązanie).